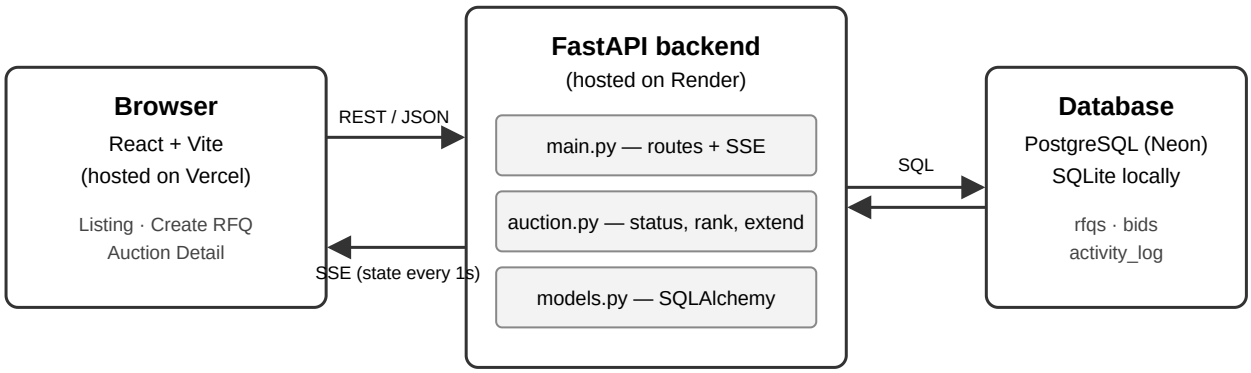
High-Level Design — British Auction RFQ System

Rehan Ahmad · October 2026 · github.com/Rehan123-bash/british-auction-rfq · live: british-auction-rfq-two.vercel.app

1. Overview

A simplified RFQ (Request for Quotation) system where a buyer creates an RFQ and carriers submit quotes openly, lowering their prices to beat each other. Bidding activity close to the end of the auction automatically extends the close time, but never beyond a hard **forced close time**. British-auction behaviour is a per-RFQ option; with it off, an RFQ is a plain sealed-deadline request that closes at its bid close time. The system has three pages: an auction listing, an RFQ creation form, and an auction details page that updates live while bids come in.

2. Architecture



No background workers or cron jobs — auction status is computed from timestamps on every read.

How the pieces fit

- **Frontend** — three React pages. Normal actions (create RFQ, submit quote) are plain `fetch` calls. The details page opens an `EventSource` and replaces its state with every message, so new bids, rank changes and close-time extensions appear without refreshing.
- **Backend** — one FastAPI app. `main.py` holds the five routes. `auction.py` holds the only non-trivial logic (status, ranking, extension) so it can be unit-tested on its own. Writing a bid and extending the auction happen in a single DB transaction.
- **Database** — three tables. The same SQLAlchemy code runs on SQLite locally and PostgreSQL in production; only `DATABASE_URL` changes.
- **No timers** — the auction's status is a pure function of the current time and three stored timestamps. An auction "closes" simply because any read after the close time reports it closed. This keeps the backend stateless and safe to restart.

3. Database schema

rfqs — one row per auction, including its configuration

column	type	notes
id	int PK	
name	text	RFQ name; shown with id as RFQ-0007
bid_start_at	timestamp	
bid_close_at	timestamp	original close
current_close_at	timestamp	moves on extension
forced_close_at	timestamp	> bid_close_at; = bid_close_at for standard RFQs
service_date	timestamp	pickup / service
british_auction	bool	false = standard RFQ, no extensions
trigger_window_minutes	int, null	X
extension_minutes	int, null	Y
extension_trigger	text, null	any_bid · any_rank_change · l1_change

bids — every quote ever submitted (history is kept)

column	type	notes
id	int PK	
rfq_id	int FK → rfqs	
carrier_name	text	supplier identity
freight_charges	numeric	
origin_charges	numeric	
destination_charges	numeric	
total	numeric	sum of the three; used for ranking
transit_time_days	int	
quote_valid_until	timestamp	
created_at	timestamp	

activity_log — audit trail shown on the details page

column	type	notes
id	int PK	
rfq_id	int FK → rfqs	
type	text	`bid` or `extension`
message	text	e.g. "Extended 18:00 → 18:05 (bid received in last 10 min)"
created_at	timestamp	

A carrier's *current* bid is its most recent row in `bids`; older rows stay for the activity history.

4. API

Method	Path	What it does
POST	`/rfqs`	Create an RFQ. Validates `bid_close_at > bid_start_at` and, for British auctions, `forced_close_at > bid_close_at` plus X / Y / trigger. Sets `current_close_at = bid_close_at`.
GET	`/rfqs`	Listing page data: id, name, status, current lowest bid, current close time, forced close time.
GET	`/rfqs/{id}`	Details page data: RFQ fields, X / Y / trigger config, ranked current bids (L1, L2, ...) with all quote fields, activity log.
POST	`/rfqs/{id}/bids`	Submit a quote. Rejected unless the auction is *Active* or if the carrier's new total is not lower than its previous one. Inserts the bid, recomputes ranking, runs the extension rule, writes activity log rows — all in one transaction.
GET	`/rfqs/{id}/stream`	Server-Sent Events. Every second, sends the same payload as `GET /rfqs/{id}`. The details page uses this to stay live.

All timestamps are ISO-8601 in UTC; the browser formats them in local time.

5. British auction logic

5.1 Auction status — derived, not stored

Condition (evaluated at read time)	Status
`now < bid_start_at`	Upcoming
`bid_start_at ≤ now < current_close_at`	Active
`now ≥ current_close_at` and (standard RFQ or `current_close_at < forced_close_at`)	Closed
`now ≥ current_close_at` and `current_close_at = forced_close_at`	Force Closed

5.2 Ranking

Each carrier's most recent bid, sorted by `total` ascending → L1, L2, L3, … Computed before and after each new bid so the extension rule can compare them.

5.3 Extension rule — runs inside every bid submission

```
maybe_extend(rfq, ranking_before, ranking_after, now):
    if not rfq.british_auction:
        return                                      # standard RFQ: never extends
    window_start = rfq.current_close_at - X minutes
    if not (window_start <= now < rfq.current_close_at):
        return                                      # bid is outside the trigger window

    if   trigger == "any_bid":          fire = True
    elif trigger == "any_rank_change":  fire = ranking_before != ranking_after
    elif trigger == "l1_change":        fire = ranking_before[0] != ranking_after[0]
    if not fire:
        return

    new_close = min(rfq.current_close_at + Y minutes, rfq.forced_close_at)
    if new_close <= rfq.current_close_at:
        return                                      # already at forced close, cannot extend

    log(rfq, "extension", old = rfq.current_close_at, new = new_close, reason = trigger)
    rfq.current_close_at = new_close
```

- **X / Y** — only bids in the last X minutes before the current close can extend it; each qualifying bid pushes the close forward by Y minutes from the current close, so steady activity keeps the auction alive.
- **Forced close** — `min(…, forced_close_at)` is the hard stop; once the close equals it, nothing can extend further and the status becomes *Force Closed*. Every extension writes an `activity_log` row with old time, new time and the trigger that fired — the reason shown on the details page.

5.4 Worked example

Bid close 18:00, X = 10 min, Y = 5 min, forced close 18:12, trigger = *any bid*.

Time	Event	Trigger window	Result
17:55	Carrier B bids	17:50 – 18:00 ✓	close → 18:05
18:03	Carrier A bids	17:55 – 18:05 ✓	close → 18:10
18:09	Carrier C bids	18:00 – 18:10 ✓	18:15 capped → 18:12
18:11	Carrier B bids	18:02 – 18:12 ✓	no extension (at forced close)
18:12	—	—	Force Closed; bids rejected

6. Deployment

Backend: Render web service from `render.yaml` (`uvicorn main:app`; env `DATABASE_URL`, `CORS_ORIGINS`), redeploys on push. **Database**: Neon PostgreSQL; tables created and demo lanes seeded on first start. **Frontend**: Vercel build of the Vite app (env `VITE_API_URL`), with a rewrite to `index.html` for client-side routes.